

Freeing rustc's Understanding: Third-Party Rust Tooling

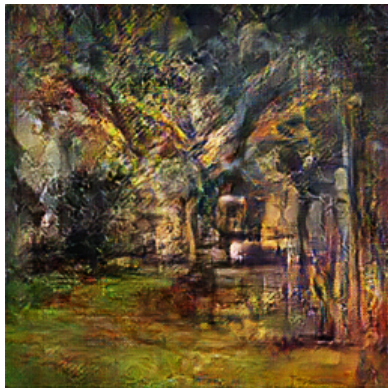
alona.page/talks/freeing-rustcs-understanding.pdf

Alona Enraght-Moony

2026-01-21

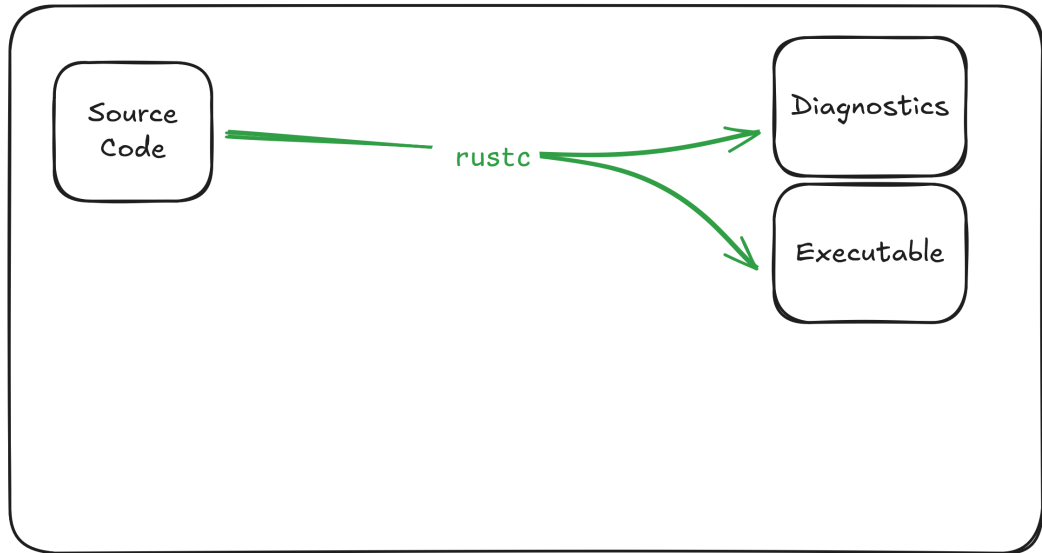
whoami (1)

- ▶ Alona Enraght-Moony (she/her)
- ▶ Rustdoc team since 2022
- ▶ Focused there on maintaining rustdoc-json
- ▶ This talk comes from soul-searching about the place that rustdoc-json fits into the ecosystem

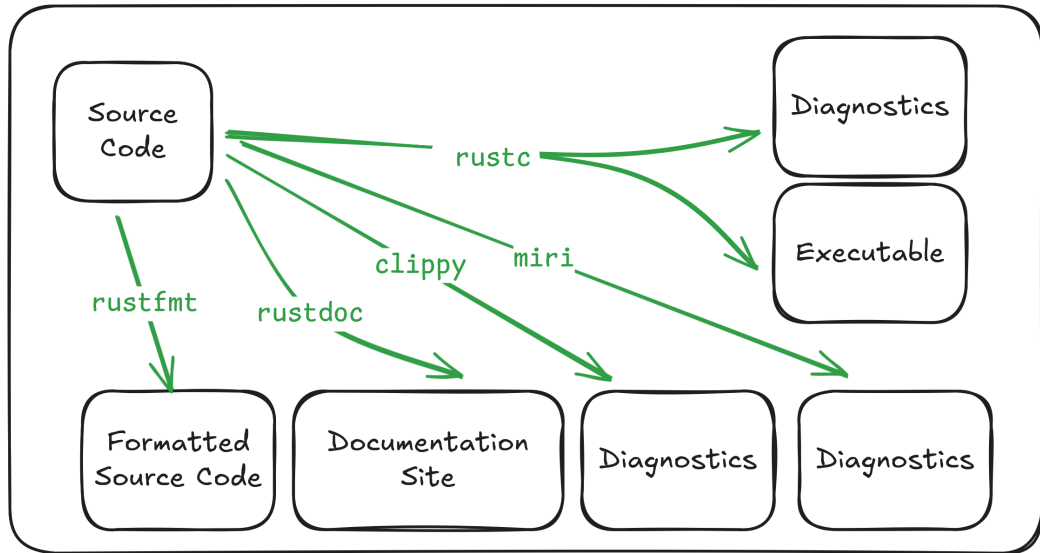


<https://github.com/aDotInTheVoid/>

rustc, the 10'000 feet view



rustc isn't enough: First party tools



The rust project isn't enough: Third party tools

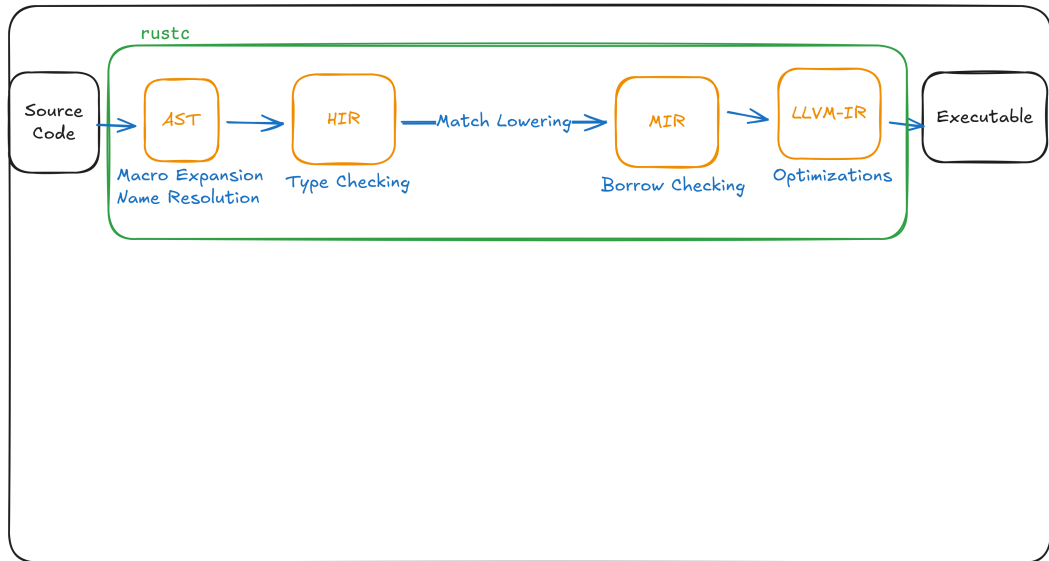
- ▶ Linters
 - ▶ cargo-check-external-types
 - ▶ cargo-semver-checks
 - ▶ klint
 - ▶ dylint
 - ▶ marker
- ▶ Verifiers
 - ▶ kani
 - ▶ aeneas
 - ▶ Creusot
- ▶ Refractoring tools
 - ▶ CoccinelleForRust
- ▶ Inspection
 - ▶ cargo-public-api
 - ▶ cargo-modules

Your tools here!

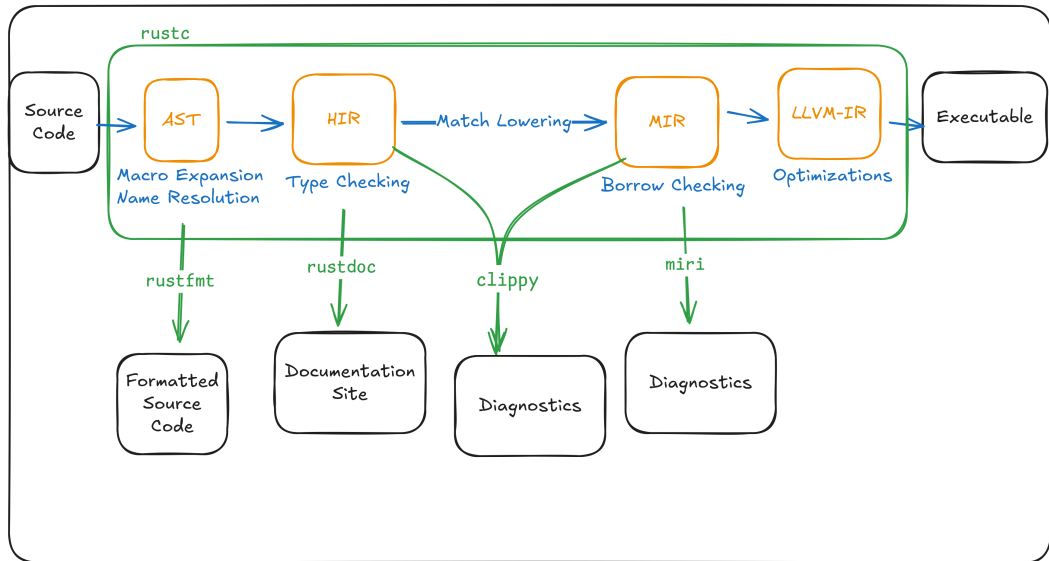
Agenda

- ▶ How tools work
- ▶ Why third-party tools are more challenging than first-party tools.
- ▶ Many different approaches to make it easier.

rustc, the 1'000 feet view



rustc, and the first party tools



Can third-party be like first party?

```
extern crate rustc_driver;
extern crate rustc_errors;

// set up target, flags, crate loader into `config`.

rustc_interface::run_compiler(config, |compiler| {
    let ast = rustc_interface::passes::parse(&compiler.sess);
    rustc_interface::create_and_enter_global_ctxt(compiler, ast, |tcx| {
        // real work here.
    })
});
```

Can third-party be like first party?

```
#![feature(rustc_private)]
extern crate rustc_driver;
extern crate rustc_errors;

// set up target, flags, crate loader into `config`.

rustc_interface::run_compiler(config, |compiler| {
    let ast = rustc_interface::passes::parse(&compiler.sess);
    rustc_interface::create_and_enter_global_ctxt(compiler, ast, |tcx| {
        // real work here.
    })
});
```

Can third-party be like first party?

```
[toolchain]
```

```
channel = "nightly-2025-02-13"
```

```
components = ["llvm-tools-preview", "rustc-dev"]
```

Can third-party be like first party?

```
$ ldd ./target/debug/demo_tool
linux-vdso.so.1 (0x00007ffff7fd6000)
librustc_driver-2fb444505ec98d04.so => /home/alona/.rustup/toolchains/nightly-2026-01-18-x86_64-unknown-linux-gnu/lib/librustc_driver-2fb444505ec98d04.so (0x00007d2cf0a00000)
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x00007d2cf86cd000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007d2cf0600000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007d2cf86c8000)
libLLVM.so.21.1-rust-1.94.0-nightly => /home/alona/.rustup/toolchains/nightly-2026-01-18-x86_64-unknown-linux-gnu/lib/./lib/libLLVM.so.21.1-rust-1.94.0-nightly (0x00007d2ce6800000)
librt.so.1 => /lib/x86_64-linux-gnu/librt.so.1 (0x00007d2cf86c1000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007d2cf86bc000)
/lib64/ld-linux-x86-64.so.2 (0x00007d2cf8723000)
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x00007d2cf0909000)
libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x00007d2cf869e000)
```

\$

Problems for third-party tools

rustc_private API breaks frequently

The screenshot shows a GitHub pull request interface for the repository 'AeneasVerif / charon'. The pull request is titled 'Update rustc #908' and is in the 'Merged' state. It was merged by 'Nadrieril' on November 25, 2025. The pull request description states: 'A small rustc bump to catch rust-lang/rust#148719 (left for a future PR). The main difficulty was rust-lang/rust#148151 which reworks how `offset_of` is represented and revealed that we didn't support inline consts properly. Most interesting changes are on the `hax` side. Of note is that `size_of` and `align_of` have become `intrinsic`s instead of `NonNull`s. Also `offset_of` might be better as a `ConstantExprKind` than a `NonNull`. I'm leaving all that to a future cleanup of builtin ops. ci: use AeneasVerif/aeneas#651 r: use AeneasVerif/aeneas#651'. The right sidebar shows no reviews, no assignees, no labels, no projects, and no milestones.

Projects have died due to this:

- ▶ gazebo_lint
- ▶ semverver

rustc_private crates are unlike any other dependency

- ▶ rustc can't be build from crates.io with cargo, but needs it's own build system
- ▶ The version of rustc that builds your tool is also the version of rustc your tool uses
- ▶ You need to be on nightly to use them
- ▶ That same version of nightly needs to be installed at runtime

How the first-party tools avoid this

rustc internal A change frequently

Don't allow rustc changes to break first party tools.

Changes to rustc must make sure that rustdoc, clippy, rustfmt, miri tests still pass.

Both of these are possible because first party tools are *developed, tested and distributed* alongside rustc.

Using rustc internals complicates the build

- ▶ First party tools are built with x.py, not cargo
- ▶ First party tools are distributed by rustup, not cargo

Summary: `#![feature(rustc_private)]`

Lets you depend access the compiler internals.

- ▶ Pros:
 - ▶ Access to all the rich state rustc has
- ▶ Cons:
 - ▶ Large API, with frequent breaking changes
 - ▶ Only usable on nightly
 - ▶ Need to dynamically link to specific version of `librustc_driver-<hash>.so`

A level of indirection: `rustc_public`

A wrapper crate implemented using `#![feature(rustc_private)]`

Currently distributes with `rustc` as an `extern crate`

- ▶ Pros:
 - ▶ More limited API, designed for tools
 - ▶ Less breaking changes
 - ▶ One day, will be on `crates.io`, support multiple `rustc` versions.
- ▶ Cons:
 - ▶ Might not have the API you need, and need to fall back to `rustc_private` crates.
 - ▶ Still need to link to version specific `librustc_driver.so`
 - ▶ `crates.io` and multi-version are still hypotheticals (as of today)

rust-analyzer as a library

De nuevo implementation of a rust compiler, for IDE support.

- ▶ Pros:
 - ▶ Freestanding
 - ▶ Usable on stable
 - ▶ Use like any other library on crates.io
- ▶ Cons:
 - ▶ API changes frequently
 - ▶ Doesn't have a "complete" compiler
 - ▶ No control-flow-graph
 - ▶ No borrow-checking
 - ▶ API designed for interactive IDE tooling
 - ▶ Optimized for latency, not throughput

Bihind (bits of) rust-analyzer: libraryification of rustc

Some bits of the compiler (e.g. `rustc_lexer` and `rustc_next_trait_solver`) are on `crates.io`, initially to be used by rust-analyzer.

► Pros:

- Freestanding
- Usable on stable
- Use like any other library on `crates.io`

► Cons:

- API's change sometimes
- Only implements the “leaf”s of analysis, not the core.
 - Bring your own data-structures:
`rustc_type_ir`
 - Draw the rest of the owl

How to write a rust compiler

1.



2.



1. depend on the librified bits of rustc

Write 'the rest of the fucking compiler

The one I worked on: `rustdoc --output-format=json`

Rustdoc feature to emit an IR to a JSON file.

- ▶ Pros:
 - ▶ Talks to rustc over a json file, not memory
 - ▶ Your tool can be in stable, but use nightly rustdoc
 - ▶ Small API, infrequent breaking changes
- ▶ Cons:
 - ▶ Tied to rustdoc, not rustc: presentational representation, not semantic
 - ▶ Only describes API, nothing about the contents of functions
 - ▶ Can't request the information you need, get all the information whether you care or not

JSON Emitter but with function bodies: Charon

A third-party tool implemented with `#![feature(rustc_private)]` that emits an IR to a JSON file.

- ▶ Pros:
 - ▶ Talks to rustc over a json file, not memory
 - ▶ Your tool can be in stable, but use nightly rustdoc
 - ▶ IR optimized for semantics and verification
- ▶ Cons:
 - ▶ Focused on semantics, not syntax.
 - ▶ Charon itself requires specific nightly.
 - ▶ Can't request the information you need, get all the information whether you care or not

That's all folks!

Thanks

Jynn Nelson, Julia Ryan, David Barsky

Contact

- ▶ email: me@alona.page
- ▶ fediverse: [@adotinthevoid@hachyderm.io](https://hachyderm.io/@adotinthevoid)
- ▶ bluesky: [alona.page](https://bsky.app/profile/alona.page)